

La procedura scorre gli n elementi di A una sola volta e utilizza operazioni costanti per ogni elemento, quindi la complessità temporale è $\Theta(n)$.

Si noti i due casi estremi in cui abbiamo solo numeri positivi o solo numeri negativi:

- Solo numeri positivi: la somma massimale corrisponde alla somma totale del vettore, la somma minimale è uguale a zero, il massimo fra somma massimale e somma totale - somma massimale è uguale alla somma totale, corretto;
- Solo numeri negativi: la somma massimale è uguale a zero, la somma minimale corrisponde alla somma totale del vettore, il massimo fra somma massimale e somma totale - somma minimale è uguale a zero, corretto.

Soluzioni errate

- Una soluzione errata consiste nel raddoppiare il vettore (copiandolo due volte in un vettore di appoggio), per poi applicare l'algoritmo di Kadane su di esso. È facile rendersi conto che è sbagliato, perché nel caso di un vettore di soli positivi viene restituito il doppio della loro somma.
- Qualcuno si è accorto di questo caso particolare e ha inserito un controllo specifico. Raramente un algoritmo "*che funziona quasi sempre, tranne in questo caso particolare qui*" è veramente corretto. Infatti, con $[2, -1, 2]$ non funziona: l'algoritmo di kadane prende l'intero sottovettore raddoppiato e restituisce 6, non 4 che è il valore corretto.
- Un'altra versione sbagliata: partendo dal valore *maxHere* nell'ultima posizione, si prova a estenderlo ricominciando dall'inizio. Di nuovo $[2, -1, 2]$ non va bene, perché *maxHere* è pari a $3=2-1+2$ e estendendolo con il 2 si ottiene 5. In pratica si è considerato il vettore $[2, -1, 2, 2]$ (dove l'ultimo 2 è il primo ripreso).

Esercizio B1 – Numeri irripetibili – Punti ≥ 8

L'algoritmo proposto utilizza una tecnica di *backtracking* per generare e stampare tutti i numeri compresi tra 0 (incluso) e 10^n (escluso) in cui nessuna cifra è ripetuta. La struttura del programma si basa su una funzione principale `printNumbers()` e una funzione ricorsiva ausiliaria `pnRec()`.

La funzione `printNumbers()` inizializza due strutture principali: un array booleano *set*, che tiene traccia delle cifre già utilizzate, e un array *S*, che rappresenta il numero in costruzione. Una cosa che hanno notato in pochi è che un numero con 11 cifre senza cifre ripetute è impossibile, quindi la dimensione massima dell'array è limitata a 10. L'array *set* è inizializzato con valori **true** per indicare che tutte le cifre da 0 a 9 sono disponibili; tuttavia poi fa un ciclo da 1 a 9 inserendo tale cifra in prima posizione e segnandola come "presa", per evitare numeri che iniziano con 0; la funzione ricorsiva `pnRec()` viene quindi chiamata con $i = 2$, indicando che si inizierà a costruire il numero dalla seconda posizione. Prima di questo, viene anche stampato il valore 0 per evitare cifre ripetute e per evitare.

La funzione `pnRec()` esplora tutte le cifre disponibili per la posizione corrente i del numero. Per ciascuna cifra k , viene verificato se questa è ancora disponibile, controllando il valore corrispondente in *set*[k]. Se la cifra è disponibile, viene assegnata alla posizione corrente *S*[i] e il numero parziale *S*[1... i] viene stampato. Successivamente, la cifra viene marcata come utilizzata (*set*[k] = **false**) e la funzione ricorsiva viene chiamata per generare il resto del numero. Al termine della chiamata ricorsiva, la cifra viene rimossa dalla soluzione corrente tramite un processo di *backtracking*, ripristinando la sua disponibilità (*set*[k] = **true**).

Questo approccio assicura che ogni numero generato soddisfi il vincolo di non avere cifre ripetute. L'esplorazione di tutte le cifre in ogni posizione garantisce che siano generati tutti i numeri validi con lunghezza massima n . Per valori di n inferiori, l'algoritmo stampa tutti i numeri con meno cifre, poiché la funzione stampa il numero anche prima di raggiungere la lunghezza n .

L'algoritmo è corretto perché ogni cifra è aggiunta solo se disponibile, evitando ripetizioni, e ogni combinazione di cifre viene esplorata esattamente una volta grazie alla natura del *backtracking*.

```
printNumbers(int n)
```

```
    boolean[] set = new boolean[0...9] = {true}
```

```
    int[] S = new boolean[1...min(10,n)]
```

```
    print 0
```

```
    for k = 1 to 9 do
```

```
        S[1] = k
        set[k] = false; pnRec(n, S, 2, set)
        set[k] = true;
```

```
pnRec(int n, int[] S, int i, boolean[] set)
```

```
    print S[1...i-1]
    if i ≤ n then
        for k = 0 to 9 do
            if set[k] then
                S[i] = k
                set[k] = false
                pnRec(n, S, i+1, set)
                set[k] = true
```

Complessità L'analisi della complessità è interessante. Un'analisi grossolana ci dice che l'algoritmo è $O(10^n)$, perché a ogni chiamata ricorsiva esploro 10 possibilità. Notate che il ciclo "ulteriore" da 1 a 10 per controllare se una cifra è presente non influisce sulla complessità, perché è costante. Utilizzare una notazione O è corretto, perché quello è il limite superiore, anche se in realtà sono molto meno. Chi ha indicato questa complessità non è stato penalizzato. Scrivere invece $O(n10^n)$ per tener conto della stampa è ancora corretto, ma ancora più impreciso perché non si accorge che numeri di più di n cifre non possono essere stampati. Anche in questo caso, ho deciso di non penalizzare questa affermazione.

In realtà, tenuto conto del fatto non posso stampare più di 10 cifre se voglio evitare ripetizioni, si potrebbe dire che il costo è limitato superiormente da... 10^{10} , che per quanto grande è un valore costante! Quindi, paradossalmente, l'algoritmo ha un costo costante. Chi ha notato questa cosa ha preso un bonus.

In realtà, è possibile stimare esattamente la quantità di numeri da stampare. I numeri di esattamente $k > 1$ cifre che non contengono cifre ripetute sono $9 \cdot 9 \cdot 8 \cdot \dots \cdot n - k + 1$, ovvero $9 \cdot 9! / (10 - n)!$. Il primo 9 deriva dal fatto che la prima cifra non può essere zero; il secondo 9 deriva dal fatto che non si può ripetere la prima cifra, l'8 che non si possono ripetere le prime 2 etc. Dovendo considerare anche il valore 0, il numero di numeri a 1 cifra è pari a 10.

Quindi, il numero di numeri di esattamente k cifre che non contengono cifre ripetute (con k da 1 a 10), è:

10, 81, 648, 4.536, 27.216, 136.080, 544.320, 1.632.960, 3.265.920, 3.265.920

Il nostro algoritmo stampa tutti i numeri con k cifre o meno, quindi i numeri effettivamente stampati sono:

[10, 91, 739, 5.275, 32.491, 168.571, 712.891, 2.345.851, 5.611.771, 8.877.691]

il cui valore è ottenuto sommando i numeri di $k - 1$ cifre o meno, e i numeri di esattamente k cifre (per esempio, 739 è ottenuto da $91 + 648$). Quindi non solo l'algoritmo ha costo costante, ma il valore è anche relativamente basso.

Valutazione sbagliata della complessità Alcuni hanno scritto $O(n2^n)$. A parte non considerare che al massimo vengono stampate 10 cifre, l'errore principale è quello relativo al valore 2: non è che se non è polinomiale, allora è limitato superiormente da un esponenziale in base 2....

Soluzioni meno efficienti Una soluzione alternativa è quella di scrivere un ciclo for che scorre tutti i numeri da 0 a $10^n - 1$, ognuno dei quali viene controllato per vedere se rispetta la condizione che la cifra non sia ripetuta. Questo è molto inefficiente perché il costo della sola generazione numeri (senza la parte di controllo, che può essere scritta in tempo $\Theta(n)$) è $\Theta(n\Theta(10^n))$. In questo caso, non fare pruning è davvero una cattiva idea. Questa soluzione è stata penalizzata.

Esercizio B2 – Giardinaggio – Punti ≥ 10

Soluzione basata su algoritmo greedy Il problema può essere risolto in modo efficiente utilizzando un algoritmo greedy, che segue la strategia di ordinare i vettori P (piante) e V (vasi) in ordine crescente. Successivamente, assegniamo il vaso più piccolo disponibile alla pianta che richiede meno spazio, proseguendo in questo modo per massimizzare il numero di assegnamenti possibili.

```
int garden(int[] P, int n, int[] V, int m)
```

```

    sort(P, n)                                % Ordina le piante
    sort(V, m)                                % Ordina i vasi
    int i = 1
    int j = 1
    int count = 0
    while i ≤ n and j ≤ m do
        if P[i] ≤ V[j] then
            count = count + 1                  % Incrementa il contatore
            i = i + 1                          % Passa alla pianta successiva
        j = j + 1                             % Passa al vaso successivo
    return count

```

La complessità temporale dell'algoritmo è pari a $O(n \log n + m \log m)$, dominata dai due ordinamenti iniziali. L'iterazione su n e m richiede $O(\min(n, m))$, che è trascurabile rispetto al costo degli ordinamenti.

Per dimostrare che la soluzione greedy è corretta, analizziamo la validità della scelta: associare il vaso più piccolo disponibile alla pianta con la richiesta minima.

Consideriamo un assegnamento ottimale tra le piante e i vasi. Supponiamo, per assurdo, che la pianta con la richiesta minima $P[i_{\min}]$ non sia associata al vaso più piccolo $V[j_{\min}]$ che soddisfa tale richiesta, ma a un altro vaso $V[j]$ con $V[j] \geq V[j_{\min}]$. Inoltre, supponiamo che il vaso $V[j_{\min}]$ sia associato a un'altra pianta $P[k]$ con $P[k] \geq P[i_{\min}]$.

Sostituendo gli assegnamenti, possiamo associare $P[i_{\min}]$ con $V[j_{\min}]$ e $P[k]$ con $V[j]$. Questa modifica non riduce il numero di piante assegnate, ma mantiene lo stesso assegnamento complessivo, poiché:

- $P[i_{\min}] \leq V[j_{\min}]$ è soddisfatto, in quanto $P[i_{\min}]$ era già compatibile con $V[j_{\min}]$;
- $P[k] \leq V[j]$ è soddisfatto, in quanto $P[k] \leq V[j_{\min}] \leq V[j]$.

In altre parole, se l'assegnamento originale era ottimale, lo è anche l'assegnamento ottenuto dallo scambio appena descritto. Questo dimostra che la scelta greedy non compromette l'ottimalità e che il nostro algoritmo produce una soluzione corretta.

Soluzione basata su rete di flusso Il problema può essere risolto modellandolo come un problema di flusso massimo su una rete. Di seguito descriviamo la costruzione della rete:

• **Nodi:**

- 1 nodo sorgente s
- n nodi, uno per ogni pianta $P[i]$, dove $i = 1, \dots, n$;
- m nodi, uno per ogni vaso $V[j]$, dove $j = 1, \dots, m$;
- 1 nodo pozzo t .

per un totale di $|V| = n + m + 2 = O(n + m)$ nodi.

• **Archi:**

- n archi, dalla sorgente s a ciascun nodo pianta $P[i]$ con capacità 1;
- m archi, da ciascun nodo vaso $V[j]$ al pozzo t con capacità 1;
- $\leq nm$ archi, uno da ciascuno nodo pianta $P[i]$ a ciascuno nodo vaso $V[j]$ se e solo se $P[i] \leq V[j]$, con capacità 1;

per un totale di $|E| \leq n + m + nm = O(nm)$ archi.

Dopo aver costruito la rete, il problema diventa trovare il flusso massimo dalla sorgente S al pozzo T . Il valore del flusso massimo corrisponderà al massimo numero di piante che possono essere assegnate a vasi compatibili. Detto $k = \min\{n, m\}$, il flusso massimo è $O(k)$, perchè potremo assegnare al massimo una pianta per vaso. La complessità dell'algoritmo di Ford-Fulkerson è quindi pari $O(|f^*| \cdot (|V| + |E|)) = O(k \cdot (n + m + 2 + O(nm))) = O(k \cdot nm)$.

Soluzione basata su programmazione dinamica Nei compiti, ho visto anche una soluzione basata su programmazione dinamica con una formula simile a questa.

$$DP[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ DP[i-1][j-1] & P[i] \leq V[j] \\ \max(DP[i-1][j], DP[i][j-1]) & \text{altrimenti} \end{cases}$$

Attenzione però: questa funziona solo se P e V sono ordinati; se non lo sono, potrei associare una pianta piccola a un vaso molto grande, e successivamente non riuscire a invasare una pianta perché non sono più disponibile vasi grandi. Se ordinate correttamente, la complessità è $O(nm)$; altrimenti l'esercizio viene considerato errato.

Commento La soluzione greedy è significativamente più veloce delle altre.

Esercizio B3 – Mosse matriciali – Punti ≥ 12

Sia $DP[r][c]$ il minimo numero di mosse necessarie per andare dalla cella (r, c) alla cella (n, n) . La tabella DP può essere calcolata tramite questa espressione ricorsiva:

$$DP[r][c] = \begin{cases} 0 & r = n \text{ and } c = n \\ \min\{\min_{c+1 \leq c' \leq \min(c+A[r][c], n)} \{DP[r][c']\}, \min_{r+1 \leq r' \leq \min(r+A[r][c], n)} \{DP[r'][c]\} + 1 & \text{altrimenti} \end{cases}$$

La spiegazione è la seguente:

- **Caso base:** Se la pedina si trova nella cella (n, n) (angolo in basso a destra), non sono necessarie ulteriori mosse, quindi $DP[n][n] = 0$.
- **Caso generale:** Per ogni altra cella (r, c) , calcoliamo la mossa ottimale considerando:
 - tutte le caselle accessibili a destra, (r, c') , con $c + 1 \leq c' \leq \min(c + A[r][c], n)$;
 - tutte le caselle accessibili in basso, (r', c) , con $r + 1 \leq r' \leq \min(r + A[r][c], n)$.

Prendiamo il valore minimo tra queste opzioni e aggiungiamo 1 per la mossa attuale.

Il codice per risolvere il problema è il seguente:

```

int minSteps(int[][] A, int n)
{
    int[][] DP = new int[1...n][1...n] = {-1}
    return msRec(A, n, DP, 1, 1)
}

int msRec(int[][] A, int n, int[][] DP, int r, int c)
{
    if r == n and c == n then
        return 0
    else
        if DP[r][c] < 0 then
            DP[r][c] = +∞
            for r' = r + 1 to min(r + A[r][c], n) do
                DP[r][c] = min(DP[r][c], msRec(A, n, DP, r', c) + 1)
            for c' = c + 1 to min(c + A[r][c], n) do
                DP[r][c] = min(DP[r][c], msRec(A, n, DP, r, c') + 1)
            return DP[r][c]
        }
}

```

L'algoritmo esegue una visita ricorsiva sulla matrice A memorizzando i risultati parziali nella tabella DP per evitare calcoli ripetuti. Per ogni cella, consideriamo fino a $O(n)$ mosse possibili (a destra o in basso), rendendo la complessità totale pari a $O(n^3)$ dove:

- n^2 è il numero di celle nella matrice,
- n è il massimo numero di mosse che possono essere considerate da ogni cella.